Can a Decision Model Be the AI Inside a Database?

An experimental study of Jev and open decision models on DuckDB

24 October 2026 (part I first issued 7 October 2026)

Summary

Jev (TypeSafe AI, released 15 September 2026) is a non-generative *decision model*. It takes a piece of state and a set of typed questions (yes/no, single choice, ordinal score) and returns the answers with probabilities in one pass, at $0.042 per million input tokens. The report has two parts. **Part I** (phases 0–5) asks whether Jev can usefully be combined with a database and tests five claims that have circulated since its release. **Part II** (phases 6–14) puts open decision models that run on a laptop in Jev's place behind the same SQL functions, and fine-tunes one of them, Kev, for record matching. In part II Jev's zero-shot answers are the reference: a model is *on par* on a data set if its accuracy is at most 2 points lower, its AUC at most 0.02 lower and its Brier score at most 0.02 higher. Every pass/fail criterion was fixed before the corresponding data were collected.

Part I, Jev on DuckDB.

Claim	Criterion fixed in advance	Result
Jev can execute semantic SQL at LLM quality	Mean quality no more than 0.1 below published systems (SemBench, movie scenario)	**Supported.** 0.765, against 0.689–0.741 published and 0.745 for a cheap LLM in the same harness
It does so far more cheaply	Measured, no threshold	**3× cheaper and 5× faster** than a cheap LLM in the same harness, not the 100× of the announcements
"Jev first, LLM when unsure" gives LLM quality at low cost	Escalating 30% of rows recovers at least 60% of the accuracy gap	**Task-dependent.** Holds for record matching (two data sets); fails for sentiment and aspect predicates
Jev's probabilities make semantic aggregates cheap	At least 3× fewer labels than sampling; 95% intervals cover at least 90%	**Not met.** 2.6× and 94.5%; an open model's token probabilities do equally well
Jev can give the optimizer selectivities for ordering semantic predicates	Median q-error at most 2; an LLM at least 2 points better than Jev to escalate to	**Not worth building.** Best and worst orders differ by 11–27%; packing the predicates into one request matches the best order

The two largest gains measured do not depend on the model at all: rewriting a pairwise semantic join as "judge each row once, then join relationally" (8,128 model calls reduced to 128, with a higher F1), and packing all predicates on a row into one request (20× less wall time for a 21-attribute profile).

Bottom line, part I. Jev is currently the cheapest and fastest way to get a per-row probability inside SQL, and that is the right way to think of it. It is not more accurate than a cheap LLM, and it offers no capability that an open-weight model's token probabilities lack.

Part II, open decision models on DuckDB.

Question	Criterion fixed in advance	Result
Can an open model take Jev's place unchanged? (phase 6)	On par on all three data sets	**No.** Kev-4B, Kev-9B and Clef-flash are on par on movie reviews, below on restaurant aspects; on product matching only Kev-9B is on par
What does a local model add inside DuckDB? (phase 7)	Later questions on the same documents at least 3× faster, identical answers	**Met for long documents only:** 43× (Kev-4B) and 23× (Kev-0.8B); 1.7–1.9× on short-text tables
Should a semantic join put a record's candidates into one request? (phases 8–9)	Packed F1 at most 1 point below pairwise; tokens and time at most 50%	**For record matching on Jev:** time 10–11%, F1 unchanged. Not on Kev-4B (time 53–59%), and not for "opposite sentiment" (F1 −8.0 points)
Does fine-tuning on public labels of other data sets bring Kev to the reference on an unseen data set? (phase 10)	On par on all four held-out data sets, no other task more than 2 points worse	**No: 2 of 4 for Kev-4B, 1 of 4 for Kev-0.8B.** Kev-4B does not improve on the publication data set
Why not on the publication data set? (phase 11)	F1 above phase 10's checkpoint	Training data of one kind: adding another publication data set or three other domains raises F1 from 0.742 to 0.962 and 0.945; rewording the question does nothing
How many labels of the data set itself are needed? (phase 12)	Smallest budget that is on par	0, 0, 100 and 300 on the four data sets; at 300 on par on all four
Do close verdicts survive another training seed? (phases 13–14)	The same verdict for seeds 1–3	3 of 4; Kev-0.8B's 2-point drop on movie reviews belongs to seed 1

Bottom line, part II. Open decision models run inside DuckDB at no marginal cost and keep the data on the machine. Untrained, they meet the reference on some tasks only and run at 4–25% of the Jev API's throughput on a laptop. For record matching a short fine-tune changes the quality picture: with about 300 labels of the data set itself, Kev-4B is on par with the reference on all four data sets tested, after 15–20 minutes of training on a laptop.

1 Questions and prior work

The study started from three questions: can Jev's strengths be combined with a database; can the "AI" in AI-for-databases be Jev; and are the widely repeated claims about it (above all, that *semantic-operator cascades* are its most valuable use) true?

Two pieces of prior work frame the answers. JEVDB [1], posted a week before this study, builds semantic filter, join, classification and ranking operators on Jev and reports 89× lower latency and 13.5× lower cost

than LOTUS [3] on SemBench [2]. It lists selectivity estimation, predicate ordering, calibration analysis and open-weight alternatives as future work; those are the parts tested here. My own earlier experiments had shown that Jev cannot choose query plans zero-shot (AUC 0.33–0.52 on the Join Order Benchmark), that it matches a supervised matcher on product data, and that a Jev-then-Opus cascade reached Opus quality at 19% of the cost on one product-matching data set.

Part II follows from the fact that Jev is closed and API-only. Several open-weight models accept the same request and return probabilities as well: Kev [7] in 0.8B, 4B and 9B versions, built on Qwen3.5 with an adapter and a decision head; Clef-flash; and two Laya models. Their published quality figures are all self-reported. Part II asks what they do in Jev's place, what running locally adds, and how far Kev's own fine-tuning path (`kev.train --init_from`) goes for the task where the gap was largest.

2 Setup

System. About 500 lines of Python register Jev as vectorised functions in DuckDB 1.5.6: `jev_noul` (probability of yes), `jev_choice`, `jev_score` and `jev_nouls` (several yes/no questions in one request). Each batch is deduplicated, sent concurrently, cached in SQLite by (model and configuration, state, question) and metered. The same functions run against an LLM by swapping the backend, and a cascade variant escalates unsure rows to a second backend.

Models. Jev 1.13.0 through the official API. LLMs through OpenRouter with the provider pinned on every request, because unpinned requests to one model landed on five different providers in six calls. The LLM used as the upper tier was chosen by a rule written before the candidates ran: on 637 product pairs where Jev is unsure, take the cheapest of eight routes within 3 points of Opus 5.5 (86.7%). Only GPT 6 Luna with low reasoning qualified (84.5%; Jev itself 74.9%). DeepSeek V4.1 Flash, DeepSeek V4 Pro and Qwen 3.8 Flash were run with thinking off to read token probabilities, and Opus 5.5 served as a stronger upper tier in the last experiment.

Data. (i) SemBench movie scenario [2]: 2,000 reviews with its ten queries, plus a 16,000-review set drawn by its own generator (15,388 distinct reviews). The truth is the critic's verdict. (ii) MAMS [4]: 3,949 restaurant-review sentences; 7 aspects × {mentioned, negative, positive} give 21 predicates with selectivities from 1% to 73%. (iii) Four record-matching test sets [5] and two end-to-end matching sets from the earlier work.

Part II models and hardware. Kev-4B, Kev-9B and Kev-0.8B (weights `jaredpalmer/kev-*@v1.0`), Clef-flash and two Laya models, all served locally on one MacBook Pro (M3 Max, 64 GB) through their own servers, sandboxed without network access. Fine-tuning used Kev's own trainer (LoRA, continued from the released checkpoints, full-precision weights on the Mac GPU). Jev's column in part II reuses its answers collected through the official API on 5 October 2026; no Jev output was used to train, fine-tune, calibrate or pick a threshold for any open model.

Part II data. The part I data sets (movie reviews, restaurant sentences), the four Magellan record-matching data sets [5] (abt-buy, amazon-google, walmart-amazon, dblp-acm; 1,916–2,473 test pairs each), the raw abt-buy and dblp-scholar tables, and for phase 11 four further Magellan data sets (beer, fodors-zagats, itunes-amazon, dblp-googlescholar). All labels are human labels of these public benchmarks.

Protocol. Prompts are first drafts written from the task descriptions and were never edited after seeing labels. Primary analyses, success criteria and stop rules were written down, with checksums of the scripts, before each run (`experiments/database/protocol/` in the repository). Timings come from cold runs; every judgment is

cached once and all later analyses replay the cache.

3 Results, part I: Jev on DuckDB (phases 0–5)

3.1 Semantic SQL on DuckDB

All ten SemBench movie queries run as plain SQL. Table 1 gives quality by query, scored with SemBench's own evaluator; Figure 1 gives totals. Jev was run cold three times with almost no variation between runs.

Table 1. SemBench movie scenario, 2,000 reviews. F1 and correlation: higher is better. Relative error: lower is better.

Query	Metric	Jev	LLM	Cascade	Published (4 systems)
Q1 five positive reviews	F1	1.000	1.000	1.000	0.952–1.000
Q2 five positive, one movie	F1	0.600	0.600	0.600	0.923–1.000
Q3 count positive	rel. error	0.214	0.214	0.143	0.357–0.571
Q4 positive ratio	rel. error	0.214	0.143	0.143	0.357–0.571
Q5 ten same-sentiment pairs	F1	0.900	0.800	0.800	0.316–1.000
Q6 ten opposite pairs	F1	0.600	0.400	0.400	0.632–0.850
Q7 all opposite pairs	F1	0.732	0.769	0.772	0.202–0.695
Q8 count by sentiment	rel. error	0.040	0.000	0.040	0.081–0.323
Q9 score each review	Spearman	0.795	0.763	n/a	0.748–0.780
Q10 rank movies	Spearman	0.412	0.421	n/a	0.404–0.438

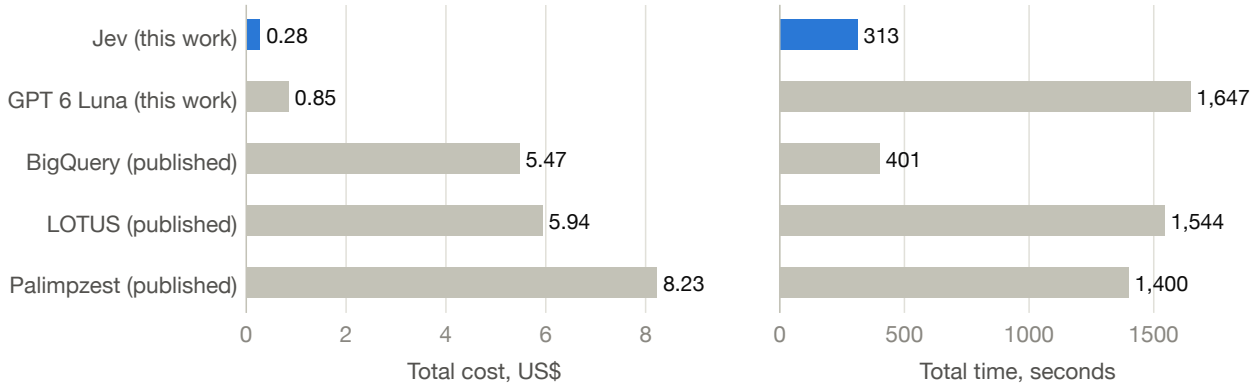


Figure 1. Total cost and time for the ten queries. Published figures were measured in 2025 with Gemini 2.5 Flash on other infrastructure, so only the first two rows are a like-for-like comparison.

Mean quality (SemBench's aggregation) is 0.765 for Jev, 0.745 for the LLM, and 0.741, 0.689 and 0.721 for BigQuery, LOTUS and Palimpzest. Against the LLM in the same harness Jev is 3.0× cheaper and 5.3× faster. Two rewrites mattered more than the choice of model. Q5–Q7 ask for pairs of reviews; judging each of the 128 distinct reviews once and joining relationally replaces 8,128 pairwise calls and raises F1 on Q7 from 0.73 to 0.79, bringing all ten queries to 90 seconds and $0.07. Returning the k most probable rows instead of the first k found (a variant added after seeing Q2) gives F1 1.0 on all four LIMIT queries.

3.2 When does a cascade work?

This is the claim the study set out to exploit, and the answer is the main finding. A cascade needs two things: an upper tier that is clearly more accurate than Jev, and Jev errors that sit in the rows it is unsure about. Record matching has both. The movie data lack the first, the restaurant data lack the second.

Table 2. Cascades by data set. "Recovered" is the share of the Jev-to-upper-tier accuracy gap regained by escalating the least-sure 30% of rows (random choice recovers 30%; the criterion was 60%). Brackets are 95% bootstrap intervals.

Data	Upper tier	Upper tier vs. Jev (accuracy)	Outcome of the cascade
Product matching, Abt-Buy (earlier work)	Opus 5.5	+11.8 points on Jev's unsure rows	Works: F1 0.902 to 0.937, 12% of rows escalated
Product matching, Walmart-Amazon	Opus 5.5	+7.5 points on Jev's unsure rows	Works: F1 0.884 to 0.929, +0.045 [0.016, 0.074], 10% escalated
Movie reviews (15,388)	GPT 6 Luna	−1.5 points overall	Fails: 18–63% of rows escalated for no gain
Movie reviews (1,851)	Opus 5.5	+1.0 point overall [−0.1, 1.9]	Nothing to gain: below the 2-point bar
Restaurant reviews (546 × 21)	GPT 6 Luna	+4.0 points [3.4, 4.5]	Fails: recovered 38% [32, 46]
Restaurant reviews (546 × 21)	Opus 5.5	+3.3 points [2.7, 3.9]	Fails: recovered 36% [28, 46]

On the movie data even Opus is not meaningfully better than Jev against the human labels, so there is nothing for a cascade to gain; the labels cap every model. On the restaurant data stronger models are better by 3–4 points, yet Jev's probabilities do not locate its mistakes. Figure 2 shows why, without involving any upper tier: on matching data 91–100% of Jev's errors lie in its least-sure 30% of rows, against 80% for movie reviews and 67% for restaurant predicates. Its mistakes there are confident ones, which follow from a mismatch between the wording of the question and the labelling convention rather than from uncertainty.

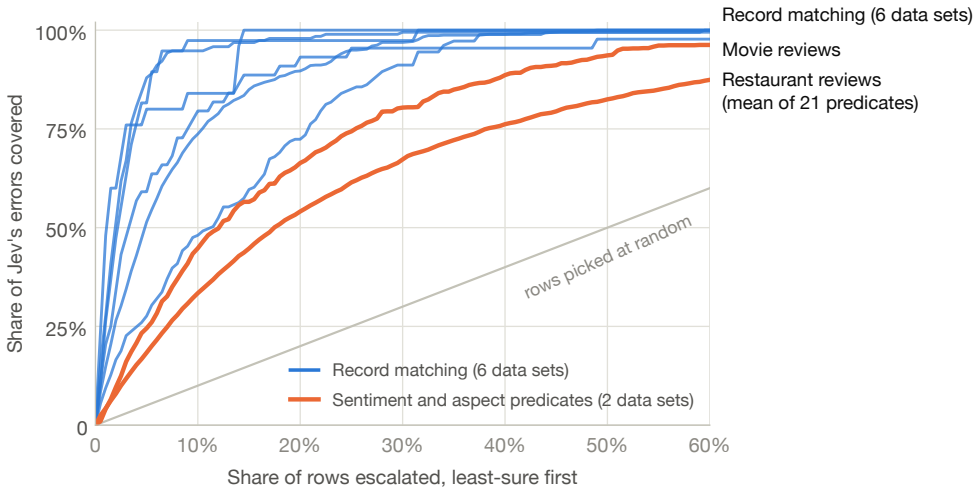


Figure 2. Share of Jev's errors that fall in the rows it is least sure about. This is the most any cascade could recover with a perfect upper tier.

A practical consequence is that this curve can be drawn from a few hundred labels before any expensive model is called, so it can serve as a test of whether a cascade is worth building for a given predicate.

3.3 Semantic aggregates

The task is to estimate the share of positive reviews among 15,388 (truth 81.3%). Without labels every model is biased: Jev thresholded 75.4%, Jev mean probability 70.0%, the LLM on every row 72.9%. Table 3 adds 200 human labels.

Table 3. Estimating a share with 200 labels (2,000 random draws each).

Method	RMSE (points)	95% interval coverage	Label saving vs. sampling
Sampling alone	2.73	95.2%	1.0×
Jev probability + difference correction (primary)	1.72	94.5%	2.5× (theory 2.62×)
Stratified sampling on Jev probability	1.38	95.5%	3.9×
LLM on every row + difference correction	2.18	94.8%	1.6×

The pre-declared 3× was not reached; coverage was. Stratifying on Jev's probability does reach 3.6–4.1×, but it was not the primary estimator. The approach is most useful for GROUP BY: calibrating Jev's probabilities on 200 labels gives a mean error of 3.7 points per movie over 87 movies, against 17.7 points when the same labels are spread across the groups. On the restaurant predicates the gain almost disappears (1.3× with a regression estimator, below 1× with the difference estimator), because Jev's answers correlate less with those labels.

3.4 Is the probability specific to Jev?

No. Table 4 swaps the source of the per-row judgment.

Table 4. Per-row judgment sources on 5,001 movie reviews. Agreement is with the human labels.

Source	Agreement	AUC	Label saving	Cost / 1,000 rows	Latency
Jev probability	91.5%	0.972	2.5×	$0.013	0.29 s
DeepSeek V4.1 Flash token probability	91.5%	0.975	2.2–2.6×	$0.060	0.42 s
DeepSeek V4 Pro token probability	89.3%	0.965	1.8–2.2×	$0.100	0.92 s
Qwen 3.8 Flash token probability	86.7%	0.968	1.9–2.1×	$0.030	2.16 s
GPT 6 Luna, yes/no only	89.6%	n/a	1.5–2.0×	$0.036	2.49 s

On the 21 restaurant predicates mean AUC is 0.871 for Jev and 0.863 for DeepSeek V4.1 Flash. Reading token probabilities from an open model with thinking switched off works as well as Jev; with thinking on, those probabilities collapse to 0 or 1 and are useless. Jev's edge is price and latency.

3.5 Selectivity, ordering and packing

Selectivity. From a 50-row sample, Jev's estimate is within a median factor of 1.13 of the truth on 87 movie subsets (a constant 1/3, PostgreSQL's default for a Boolean function, is off by 2.67). On the 21 restaurant predicates the median factor is 2.10, above the stop line of 2. Jev estimates how many rows *it* would pass (median factor 1.19), which is enough for costing a plan but not for predicting result size when wording and labels disagree.

No tier to escalate to. Asked one question at a time, the LLM beats Jev by 0.1 points on the restaurant predicates (84.1% against 84.0%), below the 2-point bar, so the escalation-ordering experiment was not built.

Ordering versus packing. Over 1,540 conjunctions of two or three predicates executed by Jev alone, the worst order costs a median 1.11× (two predicates) and 1.27× (three) of the best; ordering by Jev's sampled pass rates is within 1.00×; packing the predicates into one request costs 0.97–1.04× of the best order in tokens with 11% fewer requests, and needs no statistics.

Table 5. One question per request versus 21 questions in one request, 1,000 restaurant sentences.

Model	Accuracy, single	Accuracy, packed	Latency per request	Cost per 1,000 sentences
Jev	84.0%	84.1%	0.29 s to 0.29 s	$0.284 to $0.038
GPT 6 Luna	84.1%	88.0%	2.27 s to 3.51 s	$0.801 to $0.220
DeepSeek V4.1 Flash	86.5%	88.6%	0.45 s to 0.76 s	$1.329 to $0.477

Jev's packed answers agree with its single answers 99.5% of the time at unchanged latency; profiling the whole table took 76 seconds instead of 1,561. Packing is not specific to Jev, however. The LLMs can do it too, and they become 2–4 points *more* accurate, because seeing all the questions helps them separate neighbouring aspects. Packed, Jev remains the cheapest by a factor of 6 to 12.

4 Results, part II: open decision models as the backend (phases 6–9)

4.1 The open models unchanged (phase 6)

What was done. The five open models that run on the laptop received exactly the requests Jev had answered, with states and questions unchanged to the letter, and were judged against Jev's answers by the on-par rule. **Data.** 5,000 movie reviews ("is the review positive?"), 1,000 restaurant sentences × 21 predicates, and the 2,049 pairs of the walmart-amazon test set; 29,249 requests per model.

Table 6. Accuracy per data set and verdict against Jev. Both Laya models were below on all three and are left out.

Model	Movie reviews	Restaurant predicates	Product matching
Jev (reference)	91.5%	84.0%	97.9%
Kev-4B	90.7%, on par	82.9%, below (AUC −0.021)	93.9%, below
Kev-9B	91.6%, on par	79.4%, below	96.5%, on par
Clef-flash	90.4%, on par	81.9%, below	95.8%, below

The gap is mainly the probability scale: AUC differs from Jev's by only 0.002–0.030, and after recalibration on 200 human labels the three are within 1.4 points of Jev's accuracy. On product matching the open models call more non-matching pairs a match (Kev-4B 104, Clef-flash 72, Kev-9B 54, Jev 18). Throughput on the laptop is 4–25% of the Jev API's (Kev-4B 8.2 requests per second against 52 on movie reviews).

4.2 What a local model adds inside DuckDB (phase 7)

What was done. Three things that only a local model can do were tested with Kev-4B and Kev-0.8B: reusing text the server has already read (its prefix cache), serving as the backend for the ten SemBench queries, and calibration parameters stored in a table and applied by SQL macros. **Data.** 40 long documents (one movie's reviews joined into one text, up to 24,000 characters, about 5,800 tokens; movies outside SemBench) asked three questions each; the movie and restaurant tables; SemBench as in §3.1; three matching data sets for calibration.

Table 7. Time per row from the second question on, cache on against cache off. Answers are bit-identical in both modes.

Data	Tokens per request	Kev-4B speed-up	Kev-0.8B speed-up
Movie reviews	54–55	1.76×	1.65×
Restaurant sentences	56	1.90×	1.75×
Long documents	5,768	43.3×	22.9×

Only the long-document case passes its bar. The first pass has nothing to reuse, and there Kev-4B is 12× slower than the Jev API and Kev-0.8B 2.2× slower. As a SemBench backend Kev-4B reaches a mean quality of 0.803 (Jev 0.765) but takes 8.7× as long; Kev-0.8B is 2× slower than Jev and scores 0.664. Calibration tables work in SQL and are a precondition for Kev-0.8B (matching accuracy 66–86% raw, 91–98% calibrated), but neither model reaches the reference on two of the three new matching data sets after calibration: calibration fixes the scale, not the ranking.

4.3 How a semantic join should ask (phases 8–9)

What was done. A semantic join first finds k = 10 candidates per left record with full-text search, then asks the model. Asking one request per pair was compared with putting all k candidates into one request as k yes/no questions ("Q"). **Data.** The raw abt-buy (1,081 × 1,092 records, 1,097 true matches) and dblp-scholar tables (2,616 × 64,263, 5,347 true matches); Kev-4B ran a fixed subset of 400 left records. Phase 9 applied the same test to another predicate: all 8,128 pairs of the 128 reviews of one movie, asked "do the two reviews have opposite sentiment?".

Table 8. Pairwise against packed (Q, k = 10). Time is relative to pairwise.

Backend and data	F1 pairwise	F1 packed	Time	Verdict
Jev, abt-buy	90.7%	91.6%	10–11%	holds
Jev, dblp-scholar	90.7%	91.1%	10–11%	holds
Kev-4B, abt-buy	85.6%	91.4%	53%	fails the cost bar
Kev-4B, dblp-scholar	88.5%	90.0%	59%	fails the cost bar
Jev, opposite sentiment	73.3%	65.3%	11%	fails: answers change

On Jev, one multiple-choice question per left record also holds on the nearly one-to-one abt-buy (F1 94.0% at 22% of the tokens); on Kev-4B the same form depends on the order of the candidates (F1 73.5% to 92.3% across three orders). For other predicates packing still saves money but changes the answer, so it has to be checked per predicate.

5 Results, part II: fine-tuning Kev for record matching (phases 10–14)

5.1 Public labels of other data sets (phase 10)

What was done. The four Magellan data sets were held out in turn. Kev-4B and Kev-0.8B continued training from their released checkpoints on 2,000 human-labelled pairs from each of the other three (6,000 pairs, plus 2,000 replayed records of Kev's own training set against forgetting) and were tested on the held-out data set's full test split, asked the same first-draft question as Jev. A guard required that movie reviews, restaurant predicates and Kev's own development set drop by at most 2 points against the released checkpoint. **Data.** The Magellan training splits; test splits of 1,916–2,473 pairs.

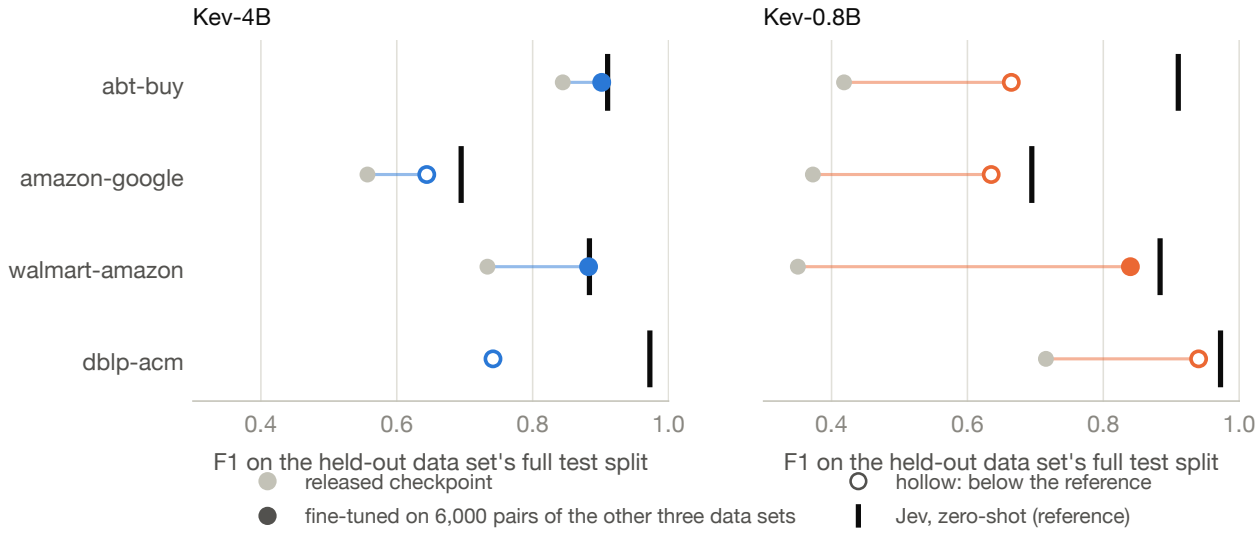


Figure 3. F1 before and after fine-tuning on the other three data sets. Filled markers meet the reference and the guard; hollow ones do not.

Kev-4B meets the reference on abt-buy and walmart-amazon and passes the guard everywhere. Most of the gain is fewer false matches (walmart-amazon: 104 to 16; Jev 18). On dblp-acm, a publication data set whose question never appears in the product training data, Kev-4B's F1 does not change (0.742). Kev-0.8B improves strongly everywhere, meets the reference on walmart-amazon, and also on dblp-acm by the three metrics, where its movie-review guard fails (−2.3 points). Trained on all four data sets, Kev-4B's F1 is above Jev's on all four test sets, but 56–93% of those test pairs have both records in the training data. End to end on the raw tables, fine-tuned Kev-4B reaches Jev's F1 (abt-buy 91.7% against 91.2%, dblp-scholar 91.0% against 90.4%).

5.2 Why the publication data set did not improve (phase 11)

What was done. Phase 10's dblp-acm checkpoint (trained on 6,000 product pairs) was the control. Three arms each added one ingredient and were tested on dblp-acm's 2,473 pairs: the product question in five equiv-

alent wordings; every training pair of three other domains with their own questions; or 2,000 pairs of another publication data set asked dblp-acm's question.

Table 9. Kev-4B on dblp-acm (444 matches).

Training data	F1	False / missed matches	On par
Jev, zero-shot (reference)	0.973	24 / 1	
6,000 product pairs (control)	0.742	26 / 167	no
+ five wordings of the product question	0.745	23 / 167	no
+ beer, restaurant and song pairs (1,156)	0.945	47 / 4	yes
+ DBLP-GoogleScholar pairs, same question (2,000)	0.962	23 / 11	yes

Rewording one question does not teach a new one. Either another publication data set or a few other domains with their own questions remove the strictness. The domains arm adds other records and other questions at once, so it cannot say which of the two does it. No arm forgets.

5.3 Labels of the data set itself (phase 12)

What was done. Each data set's records were split in two by a hash of their text: labels come only from one half, test pairs only from the other, so no test record occurs in any label. Budgets of 100, 300 and 1,000 labels were trained with the recipe Kev's author gives for one's own data (two epochs, nothing replayed; 80% of the budget trained on, 20% to fit the temperature), starting from phase 10's checkpoint and, at 300 and 1,000, from the released one. **Data.** Test sets of 632–766 pairs with 59–152 matches.

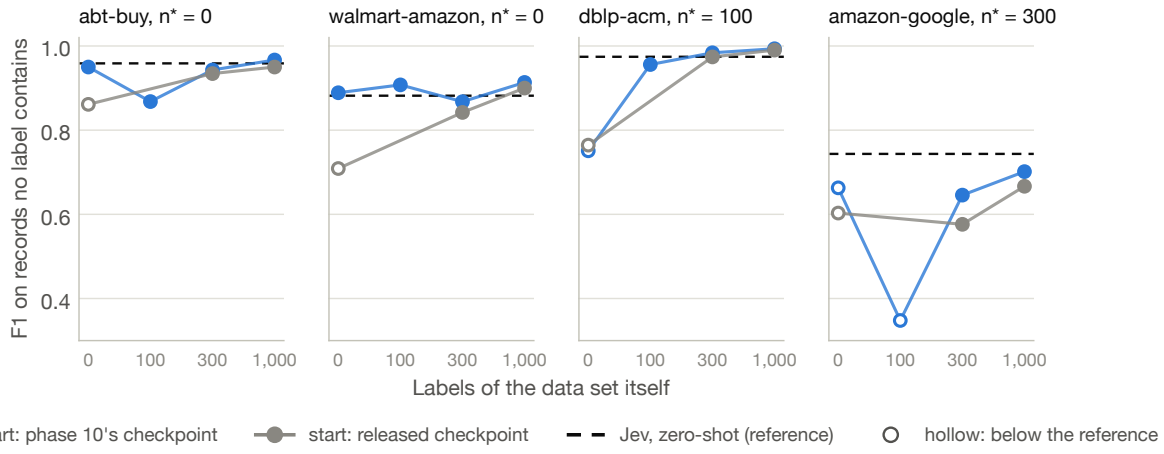


Figure 4. F1 against the number of the data set's own labels. n* is the smallest budget at which phase 10's checkpoint is on par with the reference.

No labels are needed where the data set resembles the public training data (abt-buy, walmart-amazon), 100 on the publication data set, and 300 on amazon-google, where F1 stays below Jev's (0.701 against 0.744 at 1,000) although the three metrics of the rule are met. At 300 and 1,000 labels the two starting points cannot be told apart; phase 10's public-label stage matters only with no or few labels of one's own. No run forgets.

5.4 Training seeds (phases 13–14)

What was done. Phase 10 trained every checkpoint once, with seed 1. A rule fixed in advance picked the verdicts lying within a quarter of the line's width of a line; those arms were trained again with seeds 2 and 3 (phase 13). Kev-0.8B's seed 1 was then retrained once more through the padding launcher that seeds 2 and 3 had used (phase 14), to separate the seed from that change.

Table 10. Verdicts over three seeds.

Arm	Seed 1	Seed 2	Seed 3
Kev-0.8B walmart-amazon	on par	on par	on par
Kev-0.8B amazon-google	below	below	below
Kev-0.8B dblp-acm	below (guard)	below (accuracy)	below (accuracy)
Kev-4B amazon-google	below	on par	on par

Three of four verdicts hold; Kev-4B on amazon-google turns on par with both new seeds, at the edge (Brier +0.0199 and +0.0192 against a line of +0.02). Across seeds accuracy moves by up to 1.1 points and F1 by up to 0.027. Kev-0.8B's movie-review drop was 2.0–2.4 points in all five phase 10 checkpoints and 1.7 points or less in all six with seeds 2 and 3. Seed 1 retrained through the launcher reproduces the drop (−0.0 points against phase 10's seed 1, 95% interval −0.1 to +0.1; seeds 2 and 3 are 1.3 points higher, +0.4 to +2.1): it belongs to that seed's record order and replay sample, and the launcher changes no result.

6 Discussion

Can Jev's strengths be combined with a database? Yes, in the plain sense: it runs as SQL functions, matches a cheap LLM on quality, and costs a third as much.

Can the AI in AI-for-databases be Jev? Mostly not. Supplying statistics for semantic predicates is feasible but worth little: Jev-executed predicates should simply be packed, and there was no clearly better tier whose calls were worth ordering.

Are the claims about cascades right? Only for tasks like record matching. The condition can be checked in advance with a few hundred labels (Figure 2).

The 89× of JEVDB [1] and the 3–5× measured here are not in conflict: most of the difference comes from query rewrites, de-duplication and the baseline's conditions, not from the model.

What do open decision models offer inside a database? No per-request cost, data that never leave the machine, and a cache that makes repeated questions on long documents an order of magnitude faster. Untrained, their quality is task-dependent and their throughput on a laptop is a fraction of the hosted API's. The two model-agnostic rewrites of part I still apply; packing helps a local model less, because its time grows with the tokens it reads.

How far does fine-tuning go? For record matching, labels transfer across data sets that are asked the same question, and to a new question once the training data contain more than one kind of record and question. About 300 labels of the data set itself were enough on all four data sets tested. Verdicts that lie within about a point of accuracy or 0.01 of Brier of a line need more than one training run.

7 Limitations

Two text data sets beyond matching, and one or two LLMs per role. Published SemBench cost and time figures come from a different model and year; only quality is directly comparable. The movie truth is the critic's verdict rather than the sentiment of the snippet, and the restaurant prompts visibly disagree with the annotation convention (for example "service" against "staff"), so accuracies are low for every model alike. LIMIT queries return 5–10 rows, so one row moves F1 by 0.1–0.2. LLM timings depend on concurrency and were measured once. Selectivity and ordering were evaluated by replay and not wired into DuckDB's optimizer. All data sets are public and may have been seen in training.

Part II ran on one laptop, so its speeds do not describe GPU servers, and the largest open models (Kev-27B, Clef) could not be run. Fine-tuning covered record matching only, and three of its four data sets are product data. Most checkpoints were trained once; phase 13 added two seeds for four arms only. Phase 12's test sets are small (59–152 matches), so its F1 intervals span about ±0.04–0.10, and its budgets resolve n* only to 0, 100, 300 and 1,000. Jev's part II column comes from answers collected on 5 October 2026.

8 Deviations from the pre-declared protocol

- In the qualification test 1–3% of answers from thinking routes were truncated by my token cap; the cap was raised and those rows re-asked. The selection rule was unchanged.
- A metering bug let requests that failed twice go uncounted, so 35 rows across six LLM and cascade results had been computed without an answer. A replay check found and repaired them; mean quality is unchanged to three decimals and Jev's results were unaffected.
- The stratified estimator first used n ± 1 labels; fixing this moved the simulated primary figure from 2.61× to 2.53× (theory 2.62×).
- Added after seeing results, and labelled as such: the top-k-by-probability variants, the packing comparison, and the cascade with a packed upper tier. For packing I had required the LLM's packed accuracy to be within 1 point of its single-question accuracy; it was 2–4 points higher.
- The last experiment was sized to the remaining balance. The platform reserves the full output limit of each in-flight request, which forced three collection runs; no answers were analysed between them.
- Phase 7: the first start failed before sending a request and was restarted; midway Kev-0.8B was postponed, so Kev-4B's analysis preceded Kev-0.8B's collection; one printed cost line was corrected and the report reissued with all other numbers unchanged.
- Phase 10: on the Mac GPU the trainer's memory grew with every new input length. From Kev-4B's runs on, training went through a launcher that pads inputs as Kev already does at inference; with dropout off it changes the trained weights only at rounding level, and phase 14 confirmed it changes no result. Interrupted runs were restarted from the beginning under the protocol's rules; one run was paused for 37.6 hours and resumed unchanged.
- Phase 14: before its rehearsal a training script copied from phase 13 still tested phase 13's rehearsal switch, so a rehearsal run loaded real training data; it was stopped 35 seconds in, before any training step, with nothing saved or asked.
- Phases 6, 8, 9, 11, 12 and 13 ran without deviation. All of it is recorded at the end of each protocol file.

9 Cost and reproducibility

Part I cost about $23: $19.97 on OpenRouter (the entire $20 balance) and about $3.40 on Jev. Part II cost about $3.10, all on the Jev API: its reference answers in phases 6–7 and its runs as one of the two tested backends in phases 8–9. Every open model was served and trained locally, at no cost, in about 117 hours of machine time. Code, pre-declared criteria and the commands for every table are in the project repository (`experiments/database/README.md`, `experiments/database/`, `src/kevdb/`). The cached judgments are kept locally and are not part of the repository.

10 Proposed next steps

1. **Fine-tuning beyond record matching.** Repeat phases 10 and 12 on classification and extraction predicates with human labels, to see whether the ~300-label figure carries over. Local, no API cost.
2. **A shared benchmark.** Run fine-tuned Kev and Jev on JEVDB's [1] query set, so that part II can be compared with published numbers. Needs Jev budget, roughly $5–10.
3. **Predict cascade payoff before paying for it.** Test, on six or more further predicate types, whether the error-concentration curve from 200 labels (Figure 2) predicts the measured payoff of escalation. Roughly $5 of Jev and $30–60 of a strong model.
4. **Put the model-agnostic rewrites into the optimizer.** Implement join decomposition and predicate packing as DuckDB rewrite rules and evaluate on all text-only SemBench queries.

References

1. Z. Wang, H. Yan, F. Zhao, C. Liu. *Prune First, Decide Fast: Scalable Semantic Query Processing with JEVDB.* arXiv:2610.02046, 2026.
2. *SemBench: A Benchmark for Semantic Query Processing Engines.* arXiv:2511.01716, 2025.
3. L. Patel et al. *Semantic Operators: A Declarative Model for Rich, AI-based Data Processing* (LOTUS). arXiv:2407.11418, 2024.
4. Q. Jiang et al. *A Challenge Dataset and Effective Models for Aspect-Based Sentiment Analysis* (MAMS). EMNLP 2019.
5. Y. Li et al. *Deep Entity Matching with Pre-Trained Language Models* (Ditto; Magellan benchmark splits). arXiv:2004.00584, 2020.
6. A. Angelopoulos et al. *Prediction-Powered Inference.* Science, 2023.
7. *Kev: open decision models* (model weights `jaredpalmer/kev-0.8b`, `kev-4b`, `kev-9b`, v1.0; code tag `kev-1.0`). github.com/jaredpalmer/kev.